

Software Quality Assurance

Lecture 11

Agenda

- Unit testing in Python

Unittest

Testing options

- Test first:
  - First make a sketch of all the test cases and then code them at the end
  - Cons: at the start predicting/identifying test cases is difficult at times
- Test driven
  - Define one testcase as a time
  - Code it
  - You can refactor (improve if improvement margin exists)..
  - Then move on to next test case..
- Test last: Code and then test
  - Cons…may be units are not well defined or identified
  - Bugs in units might be identified late
  - Spend most of the time in coding..and at the end less time left for testing

assertions

- The following three sets of assertion functions are defined in unittest module
- Basic Boolean Asserts
- Comparative Asserts
- Asserts for Collections
- Basic assert functions evaluate whether the result of an operation is True or False. All the assert methods accept a msg argument that, if specified, is used as the error message on failure.

Sr.No.	Method & Description
1	**assertEqual(arg1, arg2, msg = None)** Test that *arg1* and *arg2* are equal. If the values do not compare equal, the test will fail.
2	**assertNotEqual(arg1, arg2, msg = None)** Test that *arg1* and *arg2* are not equal. If the values do compare equal, the test will fail.
3	**assertTrue(expr, msg = None)** Test that *expr* is true. If false, test fails
4	**assertFalse(expr, msg = None)** Test that *expr* is false. If true, test fails
5	**assertIs(arg1, arg2, msg = None)** Test that *arg1* and *arg2* evaluate to the same object.
6	**assertIsNot(arg1, arg2, msg = None)** Test that *arg1* and *arg2* don't evaluate to the same object.
7	**assertIsNone(expr, msg = None)** Test that *expr* is None. If not None, test fails
8	**assertIsNotNone(expr, msg = None)** Test that *expr* is not None. If None, test fails
9	**assertIn(arg1, arg2, msg = None)** Test that *arg1* is in *arg2*.
10	**assertNotIn(arg1, arg2, msg = None)** Test that *arg1* is not in *arg2*.
11	**assertIsInstance(obj, cls, msg = None)** Test that *obj* is an instance of *cls*
12	**assertNotIsInstance(obj, cls, msg = None)** Test that *obj* is not an instance of *cls*

```
import unittest

class SimpleTest(unittest.TestCase):
  def test1(self):
    self.assertEqual(4 + 5,9)
  def test2(self):
    self.assertNotEqual(5 * 2,10)
  def test3(self):
    self.assertTrue(4 + 5 == 9,"The result is False")
  def test4(self):
    self.assertTrue(4 + 5 == 10,"assertion fails")
  def test5(self):
    self.assertIn(3,[1,2,3])
  def test6(self):
    self.assertNotIn(3, range(5))
```

The second set of assertion functions are **comparative asserts**

•**assertAlmostEqual** (first, second, places = 7, msg = None, delta = None)
•Test that *first* and *second* are approximately (or not approximately) equal by computing the difference, rounding to the given number of decimal *places* (default 7),
•**assertNotAlmostEqual** (first, second, places, msg, delta)
•Test that first and second are not approximately equal by computing the difference, rounding to the given number of decimal places (default 7), and comparing to zero.
•In both the above functions, if delta is supplied instead of places then the difference between first and second must be less or equal to (or greater than) delta.
•Supplying both delta and places raises a TypeError.
•**assertGreater** (first, second, msg = None)
•Test that *first* is greater than *second* depending on the method name. If not, the test will fail.
•**assertGreaterEqual** (first, second, msg = None)
•Test that *first* is greater than or equal to *second* depending on the method name. If not, the test will fail
•**assertLess** (first, second, msg = None)
•Test that *first* is less than *second* depending on the method name. If not, the test will fail
•**assertLessEqual** (first, second, msg = None)
•Test that *first* is less than or equal to *second* depending upon the method name. If not, the test will fail.
•**assertRegexpMatches** (text, regexp, msg = None)
•Test that a regexp search matches the text. In case of failure, the error message
•**assertNotRegexpMatches** (text, regexp, msg = None)
•Verifies that a *regexp* search does not match *text*. Fails with an error message

```
import unittest
import math
import re

class SimpleTest(unittest.TestCase):
   def test1(self):
      self.assertAlmostEqual(22.0/7,3.14)
   def test2(self):
      self.assertNotAlmostEqual(10.0/3,3)
   def test3(self):
      self.assertGreater(math.pi,3)
   def test4(self):
      self.assertNotRegexpMatches("Tutorials Point (I) Private Limited","Point")
```

Assert for Collections

This set of assert functions are meant to be used with collection data types in Python, such as List, Tuple, Dictionary and Set.

Sr.No.	Method & Description
1	**assertListEqual (list1, list2, msg = None)** Tests that two lists are equal. If not, an error message is constructed that shows only the differences between the two.
2	**assertTupleEqual (tuple1, tuple2, msg = None)** Tests that two tuples are equal. If not, an error message is constructed that shows only the differences between the two.
3	**assertSetEqual (set1, set2, msg = None)** Tests that two sets are equal. If not, an error message is constructed that lists the differences between the sets.
4	**assertDictEqual (expected, actual, msg = None)** Test that two dictionaries are equal. If not, an error message is constructed that shows the differences in the dictionaries.

```
import unittest

class SimpleTest(unittest.TestCase):
  def test1(self):
    self.assertListEqual([2,3,4], [1,2,3,4,5])
  def test2(self):
    self.assertTupleEqual((1*2,2*2,3*2), (2,4,6))
  def test3(self):
    self.assertDictEqual({1:11,2:22},{3:33,2:22,1:11})
```

UnitTest Framework - Skip Test

- It is possible to skip individual test method or TestCase class, conditionally as well as unconditionally. The framework allows a certain test to be marked as an 'expected failure'. This test will 'fail' but will not be counted as failed in TestResult.

```
import unittest

  def add(x,y):
    return x+y

class SimpleTest(unittest.TestCase):
  @unittest.skip("demonstrating skipping")
  def testadd1(self):
    self.assertEquals(add(4,5),9)
```

S.No.	Method & Description
1	**unittest.skip(reason)** Unconditionally skip the decorated test. *reason* should describe why the test is being skipped.
2	**unittest.skipIf(condition, reason)** Skip the decorated test if condition is true.
3	**unittest.skipUnless(condition, reason)** Skip the decorated test unless condition is true.
4	**unittest.expectedFailure()** Mark the test as an expected failure. If the test fails when run, the test is not counted as a failure.

```
class suiteTest(unittest.TestCase):
  a = 50
  b = 40

  def testadd(self):
    """Add"""
    result = self.a+self.b
    self.assertEqual(result,100)

  @unittest.skipIf(a>b, "Skip over this routine")
  def testsub(self):
    """sub"""
    result = self.a-self.b
    self.assertTrue(result == -10)

  @unittest.skipUnless(b == 0, "Skip over this routine")
  def testdiv(self):
    """div"""
    result = self.a/self.b
    self.assertTrue(result == 1)

  @unittest.expectedFailure
  def testmul(self):
    """mul"""
    result = self.a*self.b
    self.assertEqual(result == 0)
```

testsub() and testdiv() will be skipped

Test suite vs test cases vs text runner vs text fixture

- A **test case** is a class that represents an individual unit of testing. That's the place where you make sure your code works correctly. It contains fixtures and calls to assert methods to check for and report failures.
- A **test suite** is just a bunch of test cases together.
- A **test runner** is a script that takes care of running the test suite.
- **Test fixtures** provide the necessary initial conditions and configure a fixed state for software tests.
  - They also clean up after testing completes. Test fixtures provide a stable baseline for consistent, repeatable testing and allow for test initialization and clean up code to remain separate from the tests themselves.

Test suite

```
#import usertest
#import configtest # first test
import unittest # second test
class ConfigTestCase(unittest.TestCase):
        def setUp(self):
                print 'stp’
                ##set up code
        def runTest(self):
                #runs test
                print 'stp'
mySuit= unittest.TestSuite()
mySuit.addTest(unittest.makeSuite(ConfigTestCase))

runner=unittest.TextTestRunner()
runner.run(mySuit)
```

Test fixtures

- It happens that in specific scenarios, unit tests as we write them might become pure overhead. **How come?** Because we repeat the same structures, and we write the same code again and again, without really thinking about how these particular tests could become more pleasant to maintain and scale.
- Sometimes the same test fixture can apply to several tests. For example, each test in a test suite used to exercise an interface to a database may need to configure (or mock) a database connection.
- It can be time consuming or resource intensive to bring up and tear down this connection for each test. Thankfully, the *unittest framework* provides **three scopes** for test fixtures; fixtures can be defined at the **individual test, class or module level**.

- Test fixtures are methods and functions that run before and after a test.
- The intent is to provide developers hooks to set up preconditions needed for the test, and cleanup after the test.
- In many cases, this will be allocating, opening, or connecting to some resource in the setUp, and deallocating, closing, or disconnecting in the tearDown.
- The most common fixture methods are setUp and tearDown.
- The setUp() method runs before every test.
- The tearDown() method runs after every test.
- The setUp() and tearDown() methods allow you to define instructions that will be executed before and after each test method. If setUp() succeeded, tearDown() will be run whether the test method succeeded or not.

What is the difference between setUp() and setUpClass() in the Python unittest framework?

The main difference is that setUpClass is called only once and that is before all the tests, while setup is called immediately before each and every test.

setUpClass()

A class method called before tests in an individual class are run. setUpClass is called with the class as the only argument and must be decorated as a classmethod():

```
@classmethod
def setUpClass(cls):
 ...
```

The setUpClass method is for expensive elements that you would rather only have to do once, such as opening a database connection, opening a temporary file on the filesystem, loading a shared library for testing, etc. Doing such things before each test would slow down the test suite too much, so we just do it once before all the tests

```
class Example(unittest.TestCase):
        @classmethod
    def setUpClass(cls):
     print("setUpClass")
    def setUp(self):
     print("setUp")
    def test1(self):
     print("test1")
    def test2(self):
     print("test2")
    def tearDown(self):
     print("tearDown")
        @classmethod
    def tearDownClass(cls):
     print("tearDownClass")
```

When you run this test, it prints:

```
setUpClass
setUp
test1
tearDown
setUp
test2
tearDown
tearDownClass
```

Sometimes the test code can be repetitive and hence we would end up writing the same code again and again.
In this case we have a method called SetUp() method. For example let us consider the below code:

```
import unittest
class TestRepetitiveCode(unittest.TestCase):
  def test_absolute(self):
  self.var = -10
self.assertEqual(self.var, -10)

def test_overwrite(self):
  self.var = -10
self.assertTrue(self.var < 0)
```

In this case the self.var is declared in both the methods. If same variable or same piece of code is used in multiple methods then it becomes a tiring process to re-type the code again and again.

In this case we use the SetUp() method shown as below

```
import unittest
class TestRepetitiveCode(unittest.TestCase):
  def setUp(self):
  self.var = -10
def test_absolute(self):
  self.assertEqual(self.var, -10)
def test_overwrite(self):
  self.assertTrue(self.var < 0)
```

```
import unittest
def fib(n):
 return 1 if n<=2 else fib(n-1)+fib(n-2)
def setUpModule():
 print("setup module")
def tearDownModule():
 print("teardown module")
class TestFib(unittest.TestCase):
 def setUp(self):
 print("setUp")
 self.n = 10
 def tearDown(self):
 print("tearDown")
 del self.n
```

```
@classmethod
 def setUpClass(cls):
 print("setUpClass")
 @classmethod
 def tearDownClass(cls):
 print("tearDownClass")
 def test_fib_assert_equal(self):
 self.assertEqual(fib(self.n), 55)
 def test_fib_assert_true(self):
 self.assertTrue(fib(self.n) == 55)
if __name__ == "__main__":
 unittest.main()
```

output:

```
$ python test.py
setup module
setUpClass
setUp
tearDown
.setUp
tearDown
.tearDownClass
teardown module
```

To summarize:
1. setUp() and tearDown() methods are automatically used when they are available in your classes inheriting from unittest.TestCase.
2. They should be named setUp() and tearDown(), only those are used when test methods are executed.

To establish fixtures for each individual test case, override setUp() on the TestCase. To clean them up, override tearDown().

To manage one set of fixtures for all instances of a test class, override the class methods setUpClass() and tearDownClass() for the TestCase.

And to handle especially expensive setup operations for all of the tests within a module, use the module-level functions setUpModule() and tearDownModule().

Pytest

Install and get started from command prompt

- Traverse to the path:

```
C:\Users\natal\AppData\Local\Programs\Python\Python39\Scripts>
```

- Type the command: pip install -U pytest
- To invoke testing: **pytest demo.py -s**
- **-s to get print msgs displayed on console**

```
def test_square():
 n = 2
 assert n*n == 4


def test_cube():
 n = 2
 assert n*n*n == 8
```

Python Unittest vs Pytest

- Both unittest and pytest are testing frameworks in python. Unittest is the testing framework set in python by default.
- In unittest, we create classes that are derived from the unittest.TestCase module.
- It comes in handy because it is universally understood.
- Whereas using pytest involves a compact piece of code.
- Pytest has rich inbuilt features which require less piece of code compared to unittest.
- In the case of unittest, we have to import a module, create a class and then define testing functions inside the class. But in the case of pytest, we have to define the functions and assert the conditions inside them.

```
import pytest
def test_file1_method1():
x=5
y=6
assert x+1 == y,"test failed"
assert x == y,"test failed"
def test_file1_method2():
x=5
y=6
assert x+1 == y,"test failed“
```

unittest follows follows OO design. You design a class , inherit it,, the methods define test cases (as functions).
In contrast to it, pytest is available that is extensive and easy to use. It does not necessitates OO design.

Pytest Setup and Teardown - Fixtures

In Pytest, fixtures are special functions marked with the @pytest.fixture decorator.

any code before yield is the setup, and any code after yield is the teardown

```
import pytest

def calculate_square(x):
    return x * x

@pytest.fixture
def setup_data():
    data = 5  # Set up a sample number
    print("\nSetting up resources...")
    yield data  # Provide the data to the test
    # Teardown: Clean up resources (if any) after the test
    print("\nTearing down resources...")

# Test cases
def test_square_positive_number(setup_data):
    result = calculate_square(setup_data)
    assert result == 25
    print("Running test case for positive number")

def test_square_negative_number(setup_data):
    result = calculate_square(-setup_data)
    assert result == 25  # The square of -5 is also 25
    print("Running test case for negative number")
```

Markers

- Markers in Pytest
- Test functions can be marked or tagged by decorating them with 'pytest.mark.'.

- Such a marker can be used to select or deselect test functions. You can see the markers which exist for your test suite by typing

- $ pytest --markers

- @pytest.mark.skip(reason=None): skip the given test function with an optional reason. Example: skip(reason="no way of currently testing this") skips the test.

- @pytest.mark.skipif(condition): skip the given test function if eval(condition) results in a True value. Evaluation happens within the module global context. Example: skipif('sys.platform == "win32"') skips the test if we are on the win32 platform. See skipping.

- @pytest.mark.xfail(condition, reason=None, run=True, raises=None, strict=False): mark the test function as an expected failure if eval(condition) has a True value. Optionally specify a reason for better reporting and run=False if you don't even want to execute the test function. If only specific exception(s) are expected, you can list them in raises, and if the test fails in other ways, it will be reported as a true failure. See Skipping.

- @pytest.mark.parametrize(argnames, argvalues): call a test function multiple times passing in different arguments in turn. argvalues generally needs to be a list of values if argnames specifies only one name or a list of tuples of values if argnames specifies multiple names. Example: @parametrize('arg1', [1,2]) would lead to two calls of the decorated test function, one with arg1=1 and another with arg1=2. See Parametrize for more info and examples.

- @pytest.mark.usefixtures(fixturename1, fixturename2, ...): mark tests as needing all of the specified fixtures. See Fixtures

Registering Markers

- Since pytest version 4.5 markers have to be registered.They can be registered in the init file pytest.ini, placed in the test directory. We register the markers 'slow' and 'crazy', which we will use in the following example:
- [pytest] markers = slow: mark a test as a 'slow' (slowly) running test

```
# content of fibonacci.py
def fib(n):
    old, new = 0, 1
    for i in range(n):
        old, new = new, old + new
    return old
def rfib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return rfib(n-1) + rfib(n-2)
```

The corresponding test file:

```
""" content of test_fibonacci.py """
import pytest
from fibonacci import fib, rfib
def test_fib():
    assert fib(0) == 0
    assert fib(1) == 1
    assert fib(34) == 5702887
@pytest.mark.slow
def test_rfib():
    assert fib(0) == 0
    assert fib(1) == 1
    assert rfib(34) == 5702887
```

Let's start all tests, which are not marked as slow:

$ pytest -svv -k "slow"

Scenario

- Given a list of names and cell numbers, make a phonebook
- Determine the consistency of phonebook as follows:
  - If all the phone numbers are unique
  - e.g. Ali +923029191921 Asad +923029112315
  - e.g. Tahir +923029191921
  - Ali and Tahir are inconsistent

Phonebook.py

```
class PhoneBook(object):
    def __init__(self):
        self.contacts = dict()

    def add(self, name, number):
        self.contacts[name] = number

    def search(self, name):
        return self.contacts[name]

    def is_consistent(self):
        for name1, number1 in self.contacts.items():
            for name2, number2 in self.contacts.items():
                if name1 != name2 and number1 == number2:
                    return False
        return True
```

```
import unittest

class TestPhoneBook(unittest.TestCase):
    def setUp(self) -> None:
        self.phonebook = PhoneBook()

    def tearDown(self) -> None:
        super().tearDown()

    def test_phonebook_search(self):
        self.phonebook.add("Ali", "12345")
        self.assertEqual("12345", self.phonebook.search("Ali"))

    def test_phonebook_empty(self):
        with self.assertRaises(KeyError):
            self.phonebook.search("Ali")

    def test_is_phonbook_consitent_when_empty(self):
        self.assertTrue(self.phonebook.is_consistent())

    def test_is_phonbook_consitent_when_no_duplicates(self):
        self.phonebook.add("Ali", "12345")
        self.assertTrue(self.phonebook.is_consistent())

    def test_is_phonbook_consitent_when_duplicates(self):
        self.phonebook.add("Ali", "12345")
        self.phonebook.add("Tahir", "12345")
        self.assertFalse(self.phonebook.is_consistent())
```